

DSAA3073: Theories in Data Science

Tutorial 1B: From Shape to Feasible Learning

235-minute core content master · delivery adaptation pending

Wei WANG · HKUST(GZ) · 2026-08-17

Explorative projection

LESSON BACKBONE

Even when a data claim is statistically meaningful, what makes its objective optimizable and its solution computationally attainable?

Tutorial 1A gave us three items that we may now use productively: Hoeffding's inequality for bounded independent averages, the idea that an L_p norm selects a geometry, and the notation $\|x\|_p$. We will keep those meanings fixed while asking three different questions about one learning task: Is the feasible region and objective convex? Which first- or second-order certificate applies? When does a gradient step provably decrease the objective? Can the required computation finish in polynomial time?

The content-master path has four blocks: **shape before algorithm** (45 minutes), **first-order optimality certificates** (50 minutes), **curvature and a safe gradient step** (65 minutes), and **asymptotic and computational feasibility** (75 minutes). The additional definitions are shown as real content rather than hidden inside the earlier 150-minute budget. KKT, strong convexity, and full gradient-descent rate theory remain outside this foundation.

Clock	Mathematical work	Status
0–45	Convex sets, convex functions, and decisive witnesses	45 min core
45–60	Scheduled break	15 min
60–110	Differentiable, nonsmooth, and constrained first-order certificates	50 min core
110–175	Hessian curvature, smoothness, and a safe gradient step	65 min core
175–250	Big-O, encoded size, P, NP, NP-hardness, and three-axis diagnosis	75 min core
After core	Questions, slower derivations, delivery adaptation	not part of the content master

Shape before algorithm

Suppose a model is represented by $w \in \mathbb{R}^d$. A constraint such as $\|w\|_2 \leq 1$ says which parameter vectors are feasible, while an objective $f(w)$ tells us which feasible vector is preferred. These are separate objects: the set can be convex while the objective is not, or conversely.

The useful experiment is interpolation: choose two objects and inspect $\lambda x + (1 - \lambda)y$ for $0 \leq \lambda \leq 1$. A few attractive points cannot prove a statement about every pair, but one escaping interpolation can refute it. We therefore need criteria that say exactly what stays inside a feasible set and what stays below a chord.

Convex sets keep every segment

Definition: Convex set

A set $C \subseteq \mathbb{R}^d$ is **convex** if, for every $x, y \in C$ and every $\lambda \in [0, 1]$,

$$\lambda x + (1 - \lambda)y \in C. \quad (1)$$

The point $\lambda x + (1 - \lambda)y$ is a convex combination of x and y .

The quantifiers determine the proof strategy. To establish convexity, we must handle arbitrary x , y , and λ . To refute convexity, one counterexample is enough: find two points in the set and one point on their segment that leaves the set.

Worked example: An L2 ball is convex

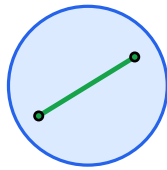
Let $C = \{w : \|w\|_2 \leq 1\}$, choose $x, y \in C$, and let $\lambda \in [0, 1]$. The triangle inequality and homogeneity give

$$\|\lambda x + (1 - \lambda)y\|_2 \leq \lambda\|x\|_2 + (1 - \lambda)\|y\|_2 \leq 1. \quad (2)$$

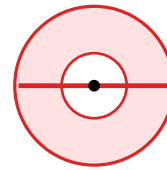
Therefore the entire segment stays in C . The same proof works for every norm ball $\{w : \|w\|_p \leq r\}$ with $p \geq 1$ and $r \geq 0$.

An annulus behaves differently. Let $A = \{w \in \mathbb{R}^2 : 1 \leq \|w\|_2 \leq 2\}$. Both $x = (1, 0)$ and $y = (-1, 0)$ lie in A , but their midpoint is $(0, 0)$, which does not. This single midpoint is a complete nonconvexity witness.

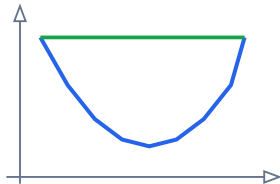
Set: segment stays inside



Set: one segment escapes



Function: graph below chord



Function: violated chord

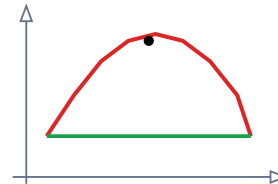


Figure 1: The same interpolation parameter governs all four panels. One escaping segment or one point above its chord refutes convexity.

IN-CLASS CHECK

Prompt: In the upper-right panel, mark two feasible endpoints and their escaping midpoint. In the lower-right panel, mark the two chord endpoints and the graph point above the chord. Write one inequality for each failure.

Hint: Use $\lambda = \frac{1}{2}$ for the pictured witnesses. A set failure is a non-membership statement; a function failure reverses the required chord inequality.

Answer:

For the annulus, $x = (1, 0)$ and $y = (-1, 0)$ are feasible but $\frac{1}{2}x + \frac{1}{2}y = (0, 0) \notin A$. For the pictured function, choose the two red endpoints a, b and their midpoint $m = \frac{1}{2}a + \frac{1}{2}b$. The black point records $f(m) < (\frac{1}{2}f(a) + \frac{1}{2}f(b))$, which is exactly a violated midpoint chord.

Convex functions lie below their chords

Definition: Convex function

Let C be a convex set. A function $f : C \rightarrow \mathbb{R}$ is **convex** if, for every $x, y \in C$ and $\lambda \in [0, 1]$,

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y). \quad (3)$$

The left side evaluates the function at an interpolated input. The right side is the height of the chord joining $(x, f(x))$ and $(y, f(y))$.

The phrase “on a convex domain” is part of the definition. A convex feasible set says interpolated **parameters** remain legal. A convex objective says the objective at those interpolated parameters lies no higher than the interpolated endpoint values. Neither statement silently supplies the other.

Worked example: A quadratic passes the chord test

For $f(w) = w^2$ on $\mathbb{R}$, expand the difference between the chord height and the function value:

$$\lambda x^2 + (1 - \lambda)y^2 - (\lambda x + (1 - \lambda)y)^2 = \lambda(1 - \lambda)(x - y)^2 \geq 0. \quad (4)$$

Hence $f(\lambda x + (1 - \lambda)y)$ is no greater than the chord height, so f is convex.

IN-CLASS CHECK

Prompt: For each object, either justify convexity or give a specific witness against it: (i) $\{w : \|w\|_1 \leq 2\}$; (ii) $\{w \in \mathbb{R}^2 : \|w\|_2 = 1\}$; (iii) $f(w) = (w - 2)^2$; (iv) $g(w) = (w^2 - 1)^2$.

Hint: For a set, test a segment. For a one-dimensional function, a violated midpoint chord is enough to refute convexity.

Answer:

(i) is convex by the norm-ball argument. (ii) is not: (1, 0) and (0, -1) are on the unit circle, but their midpoint is not. (iii) is convex because translating a quadratic does not change the nonnegative chord difference. (iv) is not convex: $g(1) = g(-1) = 0$, while $g(0) = 1$ lies above the midpoint chord height.

Key Takeaway

Convexity is a quantified interpolation statement. Proving it needs an arbitrary-pair argument; disproving it needs one decisive segment or chord. Keep the feasible-set condition separate from the objective condition.

What first-order information can certify

Convexity changes the meaning of local derivative information. For an arbitrary differentiable function, a tangent describes only nearby behavior. For a convex function, that tangent is a lower bound everywhere on the domain.

The chord inequality is now available, and derivatives are part of the course prerequisites. The next question is whether that local derivative can certify anything globally. The differentiable statement will then expose two missing cases: a convex function can have a corner, and a constrained optimum can lie on a boundary. Curvature will justify an update only after those certificates are separated.

From a chord to a tangent lower bound

Theorem: First-order condition for differentiable convex functions

Let $f : C \rightarrow \mathbb{R}$ be differentiable on an open convex set C . Then f is convex if and only if, for all $x, y \in C$,

$$f(y) \geq f(x) + \nabla f(x)^T(y - x). \quad (5)$$

Thus the tangent hyperplane at x is a global lower bound on f .

Here is the forward direction, which is the direction we will use. Fix x, y and set $z_t = x + t(y - x)$ for $0 < t \leq 1$. Convexity gives

$$f(z_t) \leq (1 - t)f(x) + tf(y). \quad (6)$$

Rearrange before taking a limit:

$$\frac{f(x + t(y - x)) - f(x)}{t} \leq f(y) - f(x). \quad (7)$$

As $t \rightarrow 0^+$, the left side becomes the **directional derivative**: the derivative at zero of the one-variable function $t \mapsto f(x + t(y - x))$. Its value is $\nabla f(x)^T(y - x)$. Therefore $f(y) - f(x) \geq \nabla f(x)^T(y - x)$. Notice the order of ideas: convexity makes the chord inequality available; differentiability turns its short-segment limit into a tangent.

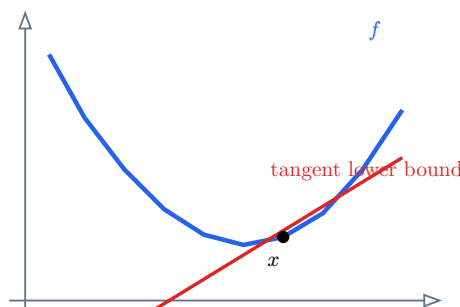


Figure 2: At the contact point the tangent and the function agree; away from it, convexity keeps the function above the tangent.

Worked example: The tangent certificate for a quadratic

Let $f(w) = \frac{1}{2}(w - 3)^2$, so $f'(w) = w - 3$. At $x = 0$ the tangent lower bound is

$$f(y) \geq f(0) + f'(0)(y - 0) = \frac{9}{2} - 3y. \quad (8)$$

Direct subtraction confirms the certificate:

$$f(y) - \left(\frac{9}{2} - 3y\right) = \frac{1}{2}y^2 \geq 0. \quad (9)$$

At $x = 0$ the derivative is negative, but the lower-bound statement concerns every y ; it is stronger than the informal phrase “the graph slopes down.”

IN-CLASS CHECK

Prompt: Suppose a differentiable convex f has $\nabla f(w^*) = 0$. Use one line to show that w^* is a global minimizer.

Hint: Insert $x = w^*$ into the first-order condition.

Answer:

For every y , $f(y) \geq f(w^*) + 0^T(y - w^*) = f(w^*)$. Therefore no point has a smaller value. This conclusion uses convexity; a zero gradient alone is not enough for a nonconvex function.

Existence, global optimality, and uniqueness are different claims

Convexity makes local certificates global, but it does not guarantee that a minimum is attained or that the minimizer is unique. The function $f(x) = e^x$ is convex on $\mathbb{R}$ and has infimum zero, yet no finite x attains that value. The constant function on $[-1, 1]$ attains its minimum at every feasible point.

Definition: Strictly convex function

A convex function $f : C \rightarrow \mathbb{R}$ is **strictly convex** if, for every distinct $x, y \in C$ and every $\lambda \in (0, 1)$,

$$f(\lambda x + (1 - \lambda)y) < \lambda f(x) + (1 - \lambda)f(y). \quad (10)$$

If a strictly convex function has a minimizer, that minimizer is unique. To see why, suppose distinct x and y both minimized f . Their midpoint would have value strictly below the average of the two minimum values, a contradiction. The phrase “if it has a minimizer” matters: strict convexity still does not prove existence.

IN-CLASS CHECK

Prompt: Classify each statement as an existence, global-optimality, or uniqueness claim:
 (i) $f(x) = e^x$ has infimum zero on $\mathbb{R}$; (ii) a convex differentiable f satisfies $\nabla f(x^*) = 0$; (iii) a strictly convex f has two distinct minimizers.

Hint: Ask whether a value is attained, whether a candidate beats every feasible point, and whether two different candidates can tie.

Answer:

(i) shows that an infimum need not be attained, so existence fails. In (ii), the first-order lower bound certifies that x^* is globally minimizing. Statement (iii) is impossible: strict convexity permits at most one minimizer, though it does not create one.

A global lower bound at a corner

Differentiability is not required for convexity. At the corner of $f(x) = |x|$, there is no unique tangent slope, but several affine lower bounds still touch the graph at zero.

Definition: Subgradient and subdifferential

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be convex. A vector g is a **subgradient** of f at $x \in \mathbb{R}^d$ if

$$f(y) \geq f(x) + g^T(y - x) \quad \text{for every } y \in \mathbb{R}^d. \quad (11)$$

The set of all subgradients at x is the **subdifferential**, written $\partial f(x)$.

For this full-space definition, when f is differentiable at x , the subdifferential contains only the gradient: $\partial f(x) = \{\nabla f(x)\}$. At a nonsmooth point it can contain many vectors. For $f(x) = |x|$,

$$\partial f(0) = [-1, 1]. \quad (12)$$

Indeed, $|y| \geq gy$ for every real y exactly when $-1 \leq g \leq 1$. In particular, $0 \in \partial f(0)$, so the subgradient inequality gives $f(y) \geq f(0)$ for every y . Thus

$$0 \in \partial f(x^*) \implies x^* \text{ is a global minimizer of convex } f. \quad (13)$$

IN-CLASS CHECK

Prompt: Find $\partial f(2)$ and $\partial f(-2)$ for $f(x) = |x|$, then explain why $g = \frac{3}{2}$ is not a subgradient at zero.

Hint: Away from the corner use the ordinary derivative. At zero test the claimed line against both positive and negative y .

Answer:

$\partial f(2) = \{1\}$ and $\partial f(-2) = \{-1\}$. If $g = \frac{3}{2}$, then for a positive y the proposed lower bound requires $|y| \geq \frac{3}{2}y$, which fails. Hence $\frac{3}{2} \notin \partial f(0)$.

A boundary optimum need not have zero gradient

Consider

$$\min_{x \geq 0} (x + 1)^2. \quad (14)$$

The minimizer is the boundary point $x^* = 0$, but $f'(0) = 2$, not zero. The unconstrained stationary rule is therefore the wrong certificate.

Theorem: First-order optimality on a convex feasible set

Let $C \subseteq \mathbb{R}^d$ be convex and let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable and convex. A feasible point $x^* \in C$ is a global minimizer of f over C if and only if

$$\nabla f(x^*)^T (y - x^*) \geq 0 \quad \text{for every } y \in C. \quad (15)$$

Necessity follows by moving a small positive distance from x^* toward any feasible y : optimality prevents a negative directional derivative. Sufficiency follows from the convex first-order lower bound,

$$f(y) \geq f(x^*) + \nabla f(x^*)^T (y - x^*) \geq f(x^*). \quad (16)$$

For the half-line example, every feasible $y \geq 0$ satisfies $f'(0)(y - 0) = 2y \geq 0$. This is a complete convex-constrained certificate without introducing Lagrange multipliers or KKT conditions.

Setting	Legal certificate	Example	Does it imply uniqueness?
differentiable convex, unconstrained	$\nabla f(x^*) = 0$	$f(x) = (x - 3)^2$	no
convex, possibly nonsmooth	$0 \in \partial f(x^*)$	$f(x) = x $	no
differentiable convex-constrained	$\nabla f(x^*)^T (y - x^*) \geq 0$ for all $y \in C$	$\min_{x \geq 0} (x + 1)^2$	no
strictly convex plus existence	any valid global certificate	a strictly convex quadratic	yes

IN-CLASS CHECK

Prompt: Match each problem to one row of the table and state whether uniqueness follows: (A) minimize $|x - 2|$ on $\mathbb{R}$; (B) minimize $(x + 1)^2$ on $[0, \infty)$; (C) minimize a constant function on $[-1, 1]$.

Hint: Choose the certificate from differentiability versus nonsmoothness and constraints; treat uniqueness as a separate question.

Answer:

(A) uses $0 \in \partial f(2)$ and has the unique minimizer 2, although that uniqueness comes from the particular function, not from the subgradient certificate alone. (B) uses the feasible-direction inequality at the boundary and has unique minimizer zero. (C) has gradient zero everywhere, so every feasible point satisfies the feasible-direction inequality; every feasible point is a minimizer, and the minimizer is not unique.

Key Takeaway

Convexity turns a gradient or subgradient into a global affine lower bound. Zero gradient is the differentiable convex, unconstrained special case; a boundary optimum uses feasible directions instead. Existence and uniqueness are separate claims, and strict convexity supplies at most one minimizer rather than guaranteeing that one exists.

Curvature and a safe gradient step

The first-order conditions can certify a minimizer once we have a candidate, but an algorithm still needs to move there. Before choosing a step size, we need two different curvature statements: a lower-curvature certificate that supports convexity or uniqueness, and an upper-curvature bound that limits how quickly the gradient can change.

Read convexity from the Hessian

Definition: Positive semidefinite and positive definite matrices

A symmetric matrix H is **positive semidefinite**, written $H \succcurlyeq 0$, if $z^T H z \geq 0$ for every z . It is **positive definite**, written $H \succ 0$, if $z^T H z > 0$ for every nonzero z .

For a twice differentiable function, the **Hessian** $\nabla^2 f(x)$ is the matrix of second partial derivatives at x .

Theorem: Second-order convexity criterion

Let f be twice continuously differentiable on an open convex domain. Then f is convex if and only if

$$\nabla^2 f(x) \succcurlyeq 0 \quad \text{for every } x. \quad (17)$$

If $\nabla^2 f(x) \succ 0$ everywhere, then f is strictly convex. Positive definiteness is a sufficient strict-convexity certificate, not a necessary one.

Worked example: Why a squared-norm penalty makes a quadratic unique

For

$$J(w) = \|Aw - b\|_2^2 + \lambda \|w\|_2^2, \quad \lambda > 0, \quad (18)$$

the Hessian is

$$\nabla^2 J(w) = 2(A^T A + \lambda I). \quad (19)$$

For every nonzero z ,

$$z^T (A^T A + \lambda I) z = \|Az\|_2^2 + \lambda \|z\|_2^2 > 0. \quad (20)$$

Hence the Hessian is positive definite and J is strictly convex, even if A has dependent columns. Chapter 4 will return to this objective; here the point is only the curvature certificate.

IN-CLASS CHECK

Prompt: Classify the Hessians $\text{diag}(2, 0)$, $\text{diag}(2, -1)$, and $\text{diag}(2, 1)$ as PSD, PD, or neither. What can each classification certify about a twice differentiable objective?

Hint: Test $z^T H z$ along each coordinate direction.

Answer:

$\text{diag}(2, 0)$ is PSD but not PD, so it can certify convexity but not strict convexity by this criterion. $\text{diag}(2, -1)$ is not PSD and therefore cannot satisfy the convexity criterion. $\text{diag}(2, 1)$ is PD and certifies strict convexity.

Upper curvature and the descent lemma

Definition: L-smooth function

A differentiable function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is **L-smooth** in the Euclidean norm if $L > 0$ and

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2 \quad \text{for every } x, y. \quad (21)$$

Smoothness is an upper bound on how quickly the gradient changes. It is not strict or strong convexity: a constant function is smooth, but it has no positive curvature lower bound and no unique minimizer.

Theorem: Descent lemma

If f is L -smooth, then for every x, y ,

$$f(y) \leq f(x) + \nabla f(x)^T(y - x) + \frac{L}{2}\|y - x\|_2^2. \quad (22)$$

Consequently, for the step $y = x - \eta \nabla f(x)$,

$$f(x - \eta \nabla f(x)) \leq f(x) - \eta \left(1 - L\frac{\eta}{2}\right) \|\nabla f(x)\|_2^2. \quad (23)$$

Thus $0 < \eta \leq \frac{1}{L}$ is a simple sufficient range for a non-increasing objective, with strict decrease whenever the gradient is nonzero. This is a one-step guarantee, not a complete convergence-rate theorem. It corrects the vague claim that moving against a gradient must always decrease the objective.

The unconstrained gradient method

Definition: Gradient method (gradient descent)

For a differentiable objective $f : \mathbb{R}^d \rightarrow \mathbb{R}$, an initial point w_0 , and a positive step size η , the unconstrained gradient method updates

$$w_{t+1} = w_t - \eta \nabla f(w_t). \quad (24)$$

When this update is used to minimize an objective, the standard textbook name is **gradient descent**. The gradient $\nabla f(w_t)$ is the vector of partial derivatives at the current iterate. The negative sign moves against the local direction of greatest increase. This Tutorial does not project the update onto a constraint set.

The rule is a state transition, not a convergence theorem. To execute it we must know the current iterate, evaluate the current gradient, declare η , and then compute the next iterate. The descent lemma supplies a sufficient decrease condition when an L -smoothness bound is available. Without such a condition, the formula alone is not a guarantee.

Worked example: Two updates, with every state visible

Continue with $f(w) = \frac{1}{2}(w-3)^2$, start at $w_0 = 0$, and take $\eta = \frac{1}{2}$. Here $f''(w) = 1$, so the gradient is 1-Lipschitz and $\eta = \frac{1}{2} \leq \frac{1}{L}$ lies in the sufficient descent range. Since $f'(w) = w-3$,

$$f'(w_0) = -3, \quad w_1 = 0 - \left(\frac{1}{2}\right)(-3) = \frac{3}{2}, \quad (25)$$

$$f'(w_1) = -\frac{3}{2}, \quad w_2 = \frac{3}{2} - \left(\frac{1}{2}\right)\left(-\frac{3}{2}\right) = \frac{9}{4}. \quad (26)$$

The objective values are $f(w_0) = \frac{9}{2}$, $f(w_1) = \frac{9}{8}$, and $f(w_2) = \frac{9}{32}$. Each step halves the distance to the minimizer $w^* = 3$ and quarters the objective gap.

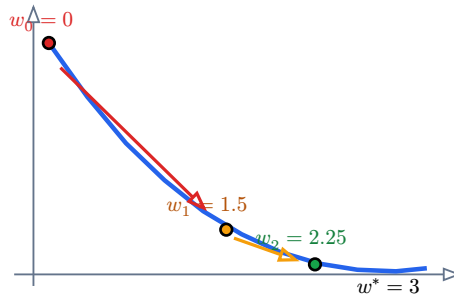


Figure 3: For $\eta = \frac{1}{2}$, the three iterates follow one fixed quadratic. Arrow lengths show iterate displacement, not gradient magnitude.

Why the step size matters

For this quadratic the error $e_t = w_t - 3$ obeys an exact recurrence:

$$e_{t+1} = w_t - \eta(w_t - 3) - 3 = (1 - \eta)e_t. \quad (27)$$

This makes four behaviors transparent.

η	Error multiplier	Behavior for this quadratic
$\frac{1}{2}$	$\frac{1}{2}$	same side, distance halves
$\frac{3}{2}$	$-\frac{1}{2}$	crosses the minimizer, distance halves
2	-1	alternates forever; objective does not decrease
$\frac{5}{2}$	$-\frac{3}{2}$	alternates with growing distance; diverges

IN-CLASS CHECK

Prompt: For $f(w) = \frac{1}{2}(w-3)^2$ and $w_0 = 0$, first identify a valid smoothness constant L . Which step sizes are certified by the displayed descent-lemma range? Verify its one-step bound for $\eta = \frac{1}{2}$. Then compute w_1 and w_2 for $\eta = \frac{3}{2}$ and $\eta = \frac{5}{2}$. Which case overshoots but converges, and which diverges?

Hint: Use $f''(w) = 1$ and then distinguish the lemma's sufficient range from the exact error multiplier $1 - \eta$.

Answer:

Here $L = 1$, so the stated sufficient range certifies $0 < \eta \leq 1$. At $\eta = \frac{1}{2}$, the lemma gives $f(w_1) \leq \frac{2}{9} - \frac{2}{9} = 0$; direct calculation gives equality. With $\eta = \frac{3}{2}$, $w_1 = \frac{2}{3}$ and $w_2 = \frac{2}{3}$; this step lies outside the displayed sufficient range, but the exact multiplier $-\frac{1}{2}$ shows convergence. With $\eta = \frac{5}{2}$, $w_1 = \frac{2}{5}$ and $w_2 = -\frac{4}{15}$; the multiplier $-\frac{3}{2}$ makes the error grow. A sufficient theorem need not characterize every convergent step size.

Key Takeaway

The Hessian supplies a lower-curvature certificate for convexity or strict convexity, while L -smoothness supplies an upper-curvature model. The descent lemma uses that upper model to justify a safe gradient step. This is a one-step guarantee; strong convexity and full convergence rates remain later topics.

Asymptotic and computational feasibility

We can now estimate a fixed predictor's mean loss and optimize a simple convex objective. A third question remains: how does the work scale with the size of the encoded input? Statistical sufficiency, favorable geometry, and efficient computation are different guarantees.

The same task may contain bounded losses, a parameter geometry, an objective, and an implementation. If the full claim fails, we should locate the broken link before reaching for a theorem: a probability bound does not count operations, and a convexity proof does not specify code. We will audit the statistical, optimization, and computational claims separately and combine only the conclusions that have evidence.

What a Big-O statement actually says

Definition: Big-O notation

Let f and g be nonnegative functions for all sufficiently large inputs. We write

$$f(n) = O(g(n)) \quad (28)$$

if there are constants $c > 0$ and n_0 such that

$$f(n) \leq cg(n) \quad \text{for every } n \geq n_0. \quad (29)$$

The constants may depend on parameters explicitly declared fixed, but they must not depend on the scaling variable n . If several variables, such as n and d , are declared to scale, the constant must be independent of all of them and the claim must hold over the stated joint input domain. One may instead express the runtime using the total encoded length N .

Big-O discards constant factors and a finite prefix of input sizes; it does not assert exact equality, a matching lower bound, or a particular leading constant. For example, for every $n \geq 1$,

$$7n^2 + 3n + 4 \leq 14n^2, \quad (30)$$

so $7n^2 + 3n + 4 = O(n^2)$ with one valid choice $c = 14$ and $n_0 = 1$.

The scaling variable cannot be silently changed.

Runtime form	Declared scaling	Valid summary	What may not be hidden
$4n + 20$	n	$O(n)$	nothing
$3nd^2 + 7d^3 + 50$	n and d	$O(nd^2 + d^3)$	d
$3nd^2 + 7d^3 + 50$	n ; fixed d	$O(n)$	the claim is conditional on fixed d
$2^d d$	d	$O(2^d d)$	the exponential factor

For nonnegative runtimes, the sequence $1, \log n, n, n \log n, n^2, n^3$, and 2^n is ordered by asymptotic growth: for each adjacent pair f, g , the ratio $\frac{f(n)}{g(n)}$ tends to zero. A polynomial

term can dominate another polynomial term, but every fixed-degree polynomial is eventually dominated by 2^n .

IN-CLASS CHECK

Prompt: Give valid constants for $5n + 12 = O(n)$, then decide whether $2^n = O(n^{10})$. Finally, classify $T(n, d) = nd + d^3$ when both parameters scale and when d is fixed.

Hint: For the first claim try $n_0 = 1$. For the exponential claim compare eventual growth, not small numerical inputs.

Answer:

For $n \geq 1$, $5n + 12 \leq 17n$, so $c = 17$ and $n_0 = 1$ work. The claim $2^n = O(n^{10})$ is false because the exponential eventually exceeds every fixed-degree polynomial by an unbounded ratio. If both n and d scale, $T(n, d) = O(nd + d^3)$. If d is explicitly fixed, its powers enter the constants and the remaining dependence is $O(n)$.

Polynomial in which input size?

Definition: Polynomial-time algorithm

After fixing a computation model and an encoding, an algorithm is **polynomial time** if its number of elementary operations is at most cN^k for constants c, k independent of the instance, where N is the number of symbols or bits in the encoded input. The relevant input parameters must be stated; hiding one of them can change the classification.

The encoding clause prevents a common mistake. Suppose an integer M is written in binary, so its representation has about $\log_2 M$ bits. A loop that runs M times is polynomial in the **numeric value** M , but exponential in its encoded length $N \approx \log_2 M$, because $M \approx 2^N$.

Worked example: Linear scan versus exhaustive enumeration

Reading N stored loss values once takes $O(N)$ operations. By contrast, evaluating every binary vector in $\{0, 1\}^d$ takes 2^d evaluations. Even if one evaluation is polynomial in d , the full enumeration is exponential in d . A faster processor changes constants; it does not turn 2^d into a polynomial.

IN-CLASS CHECK

Prompt: An algorithm receives a d -bit description and uses $O(d^3)$ preprocessing followed by $O(2^d)$ search. Is the complete algorithm polynomial time in d ? What if d is fixed once and never scales?

Hint: Classify the sum by its fastest-growing term, then distinguish a fixed constant from an input parameter.

Answer:

The complete runtime is exponential in variable d because $2^d d$ dominates every polynomial. If d is genuinely fixed outside the problem sequence, then 2^d is a constant and the remaining dependence may be polynomial in other inputs. The runtime claim must say which parameters scale.

Solving and verifying are different tasks

The runtime of an algorithm is a property of one method. Complexity classes instead organize whole **decision problems**: each encoded instance asks for a yes/no answer. For example, an optimization task may ask for the smallest decision tree, while its decision version asks whether a tree of size at most k with the required accuracy exists.

As a concrete decision problem, **LIST-MEMBER** receives a list $a_1, \dots, a_N$ and a query value v , then asks whether some $a_i = v$. A left-to-right scan decides both yes-instances and no-instances in $O(N)$ time.

Definition: Complexity class P

P is the class of decision problems that a deterministic algorithm can solve in time polynomial in the encoded input length. A yes-instance and a no-instance must both be decided within that runtime bound.

Definition: Complexity class NP

A decision language L lies in NP if there are a polynomial q and a deterministic polynomial-time verifier V such that, for every encoded instance x ,

$$x \in L \leftrightarrow \exists c : |c| \leq q(|x|) \text{ and } V(x, c) = 1. \quad (31)$$

Thus every yes-instance has a polynomial-length accepting **certificate**, while a no-instance has no certificate that V accepts. The verifier receives both the original instance and the proposed certificate; it checks the certificate rather than searching for one.

Every problem in P also lies in NP: a polynomial-time solver can be used as a verifier without needing a meaningful certificate. Thus $P \subseteq NP$. Whether equality holds is an open problem; the standard conjecture is $P \neq NP$.

Worked example: A satisfying assignment is a certificate

Consider the decision problem: “Does this Boolean formula have a satisfying assignment?” A proposed assignment has length polynomial in the formula encoding, and we can substitute its values and evaluate the formula in polynomial time. This explains why the problem belongs to NP. It does not give a polynomial-time method for **finding** a satisfying assignment and therefore does not by itself place the problem in P.

IN-CLASS CHECK

Prompt: A student says: “If I can check a proposed solution quickly, then I can find one quickly.” Which class membership has actually been justified, and what additional conclusion would be needed to place the problem in P?

Hint: Separate verification with a supplied certificate from searching for or deciding the answer.

Answer:

Polynomial-time verification justifies membership in NP only when the verifier accepts some polynomial-length certificate exactly for the yes-instances and accepts none for a no-instance. Membership in P requires a deterministic polynomial-time algorithm that decides every instance; the verifier alone does not supply such an algorithm.

Hardness comes from a reduction, not from one slow algorithm

Definition: Polynomial-time reduction

A polynomial-time reduction from decision problem A to decision problem B maps every encoded instance x of A to an instance $r(x)$ of B in polynomial time, preserving the answer: x is a yes-instance of A exactly when $r(x)$ is a yes-instance of B . Therefore a polynomial-time solver for B would give a polynomial-time solver for A .

The direction is the important part: reducing A to B shows that B is at least as hard as A . It does not show that A is at least as hard as B .

Definition: NP-hard problem

A decision problem B is **NP-hard** if every decision problem in NP has a polynomial-time reduction to B . It need not itself belong to NP.

For an optimization target, an NP-hardness statement must name the associated decision version or prove that an exact optimizer would decide an NP-hard decision problem. The optimization problem itself is not thereby a member of NP, because NP is a class of decision problems.

Object	Polynomial-time requirement	Boundary to remember
P	solve every decision instance	$P \subseteq NP$
NP	verify polynomial-length certificates for yes-instances	verification is not the same as finding
reduction A to B	compute $r(x)$ while preserving yes/no	a fast solver for B would solve A
NP-hard B	every problem in NP reduces to B	a decision target need not lie in NP

Table 1: The class relation and reduction direction. Hardness flows toward the reduction target; an algorithmic upper bound for the target flows back to the source.

Suppose A is already known to be NP-hard and we construct a polynomial-time reduction from A to B . By transitivity, every problem in NP reduces to B , so B is NP-hard. If B also had a polynomial-time exact solver, then every problem in NP would be in P, implying $P = NP$.

This statement does **not** prove an exponential lower bound. NP-hardness is a relative hardness result. Under the conjecture $P \neq NP$, it rules out a polynomial-time exact solver in the declared model, but it does not say that every heuristic is useless, that approximation is impossible, or that one particular exponential implementation is optimal.

IN-CLASS CHECK

Prompt: A known NP-hard decision problem A has a polynomial-time reduction to a decision problem B_{dec} associated with a new learning optimization target B_{opt} . What has been proved about B_{dec} ? What extra link permits us to call B_{opt} NP-hard? What would the reverse reduction—from B_{dec} to A —fail to prove? Finally, why does an observed 2^d enumeration prove none of these hardness statements?

Hint: Follow the consequence “a fast solver for the target would solve the source.”

Answer:

The reduction from known NP-hard A to B_{dec} proves that B_{dec} is NP-hard. We may call B_{opt} NP-hard when a polynomial-time exact optimizer for B_{opt} would decide B_{dec} . A reverse reduction only shows that a solver for A could solve B_{dec} ; it does not make B_{dec} NP-hard. A 2^d implementation is merely one upper bound for one algorithm. Another algorithm might be polynomial, so the runtime observation alone proves no hardness classification.

Enough data is not the same as a fast search

Definition: Computational-statistical gap

A **computational-statistical gap** occurs when the available information or sample guarantees permit the desired statistical accuracy, while no computationally efficient method is known or available under the same stated assumptions. It is a separation between what the data can identify and what the algorithm can feasibly compute.

Worked example: A qualified positive gap from one-way functions

A **one-way function** is computable in polynomial time but hard to invert on its prescribed random inputs. More precisely, the assumption says that the inversion success of every randomized polynomial-time algorithm is **negligible**: for every positive polynomial p , that success probability is below $\frac{1}{p(N)}$ for all sufficiently large encoded input lengths N . Here “randomized polynomial time” allows the algorithm to use random bits, while its prescribed running-time bound remains polynomial in N . Chapter 8 of the source considers inverse functions indexed by polynomial-length secret keys. It establishes two facts for this family: polynomially many labeled examples suffice statistically, yet a polynomial-time learner would yield an efficient inverter and contradict the one-way-function assumption.

This is a genuine but conditional computational-statistical gap. The statistical and computational claims concern the same inverse-function family, and the computational obstruction names its assumption. We use the source result as stated; Chapter 1 does not derive its sample theorem or its cryptographic reduction.

Contrast that example with a finite collection of 2^d candidate predictors. Suppose a separate statistical analysis has already supplied a polynomial sample guarantee, while the current exact implementation evaluates all 2^d candidates. This proves that **enumeration** is inefficient; it does not prove that the learning task is NP-hard. A valid NP-hardness claim needs the reduction evidence defined above. The class may have structure that supports a faster algorithm. A slow implementation is therefore a warning to look for a gap, not a positive gap example by itself.

Same task	Statistical	Optimization	Computational
Assumptions	independent losses in $[0, 1]$; chosen ε, δ	$C = \{w : \ w\ _2 \leq 1\}$; quadratic objective	encoded dimension d ; exact selector enumerates 2^d states
Guarantee	fixed-predictor mean-loss error is controlled	interpolation is feasible; tangents are global lower bounds	
Evidence status	supported by Hoeffding	supported by segment and chord checks	no polynomial runtime bound for the stated selector
Do not infer	therefore the optimizer is fast	therefore the data are sufficient	therefore the output is statistically accurate

Table 2: Three audits of the same bounded-loss pipeline. Red statements are unsupported cross-column inferences.

IN-CLASS CHECK

Prompt: Fill the missing computational guarantee cell. Then state what must change before this same pipeline can be called polynomial time.

Hint: Classify the declared 2^d selector; do not replace an absent guarantee with a hope about another algorithm.

Answer:

The missing cell is: **no polynomial-time guarantee for the stated exact selector; its runtime is exponential in d** . To obtain a positive polynomial-time claim, the team must give a different implementation and a runtime bound polynomial in all declared encoded inputs. Hoeffding and the convexity of a continuous relaxation do not supply that implementation.

IN-CLASS CHECK

Prompt: Which case is a genuine computational-statistical gap: (i) one attempted selector is exponential but no hardness result is known, or (ii) under the one-way-function assumption, polynomially many examples suffice for the inverse-function family but an efficient learner would break the assumption?

Hint: A positive instance must pair a statistical guarantee and a computational obstruction for the same task under the same conditions.

Answer:

Case (ii) is the qualified positive instance. Case (i) diagnoses only a slow implementation, an untied structured algorithm might still be efficient.

IN-CLASS CHECK

Prompt: Match each situation to the governing foundation and state one unsupported conclusion: (A) losses are bounded but observations may be dependent; (B) the feasible set contains two points but not their midpoint; (C) an exact method runs in 2^d time although the accuracy guarantee needs only polynomially many samples; (D) a convex gradient update uses an undeclared step size.

Hint: Use the words statistical, geometric, optimization, and computational only after naming the missing assumption or certificate.

Answer:

(A) is statistical: independence is missing, so the displayed Hoeffding guarantee is unsupported. (B) is geometric: the midpoint refutes convexity of the feasible set, so convex-optimization conclusions are unsupported. (C) is computational: sample sufficiency does not make enumeration efficient. (D) is optimization-related: the update can be computed symbolically, but decrease or convergence is unsupported until the step size and objective conditions are checked.

Chapter 1 boundary

Foundation	Question it answers	Representative certificate
Concentration	Can a finite average approximate an expectation?	Hoeffding with boundedness and independence
Geometry	What counts as size, direction, interpolation, and a legal parameter?	$\ w\ _p$, compact SVD, and a segment test
Convex analysis	What global statement follows from local first-order information?	a gradient, subgradient, or feasible-direction lower bound
Curvature	What certifies convexity, uniqueness, or a safe local step?	PSD/PD Hessian and an L -smooth upper model
Asymptotics and algorithms	How does the required computation scale with encoded inputs?	a quantified Big-O and polynomial runtime bound
Complexity classes	Is the decision problem efficiently solvable, efficiently verifiable, or a reduction target for every problem in NP?	$P \subseteq NP$ and a correctly directed polynomial-time reduction
Computational-statistical separation	Can samples suffice while efficient learning remains unavailable?	a same-task positive instance with its hardness assumption

Key Takeaway

A learning guarantee is only as strong as its weakest unproved link. Probability controls sampling uncertainty; norm and singular-value geometry expose directions; convexity and curvature license optimality and descent claims; and asymptotic analysis controls scalable implementation. Diagnose the missing link before choosing the tool.

Standard language to carry forward

Term	Meaning here	Representative form
Convex set	Every segment between two points remains in the set.	$\lambda x + (1 - \lambda)y \in C$
Convex function	Its value at an interpolated input lies below the endpoint chord.	$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$
Strict convexity	Every nontrivial chord inequality is strict; an attained minimizer is unique.	$x \neq y, 0 < \lambda < 1$
First-order condition	For differentiable convex f , every tangent hyperplane is a global lower bound.	$f(y) \geq f(x) + \nabla f(x)^T(y - x)$
Subgradient	A possibly nonunique slope vector gives a global affine lower bound at a convex non-smooth point.	$g \in \partial f(x)$
Constrained first-order condition	At a convex-feasible minimizer, every feasible displacement has nonnegative gradient inner product.	$\nabla f(x^*)^T(y - x^*) \geq 0$
PSD / PD Hessian	Nonnegative quadratic forms certify convexity; positive ones certify strict convexity under the stated differentiability conditions.	$\nabla^2 f(x) \succcurlyeq 0$ or $\succ 0$
L -smoothness	The gradient changes at most linearly, producing a quadratic upper model.	$\ \nabla f(x) - \nabla f(y)\ _2 \leq L\ x - y\ _2$
Descent lemma	A smooth upper model gives a sufficient decrease range for a gradient step.	$0 < \eta \leq \frac{1}{L}$
Gradient method / gradient descent	Subtract a positive step size times the current gradient to minimize an objective.	$w_{t+1} = w_t - \eta \nabla f(w_t)$
Big-O notation	An eventual upper bound up to a constant independent of the declared scaling variable.	$f(n) = O(g(n))$
Polynomial time	Runtime is polynomial in declared encoded input sizes under a fixed computation model.	$O(N^k)$
P	Decision problems solvable in polynomial time.	solve
NP	Decision problems for which a polynomial-time verifier accepts some polynomial-	$x \in L \leftrightarrow \exists c : c \leq q(x), V(x, c) = 1$

	length certificate exactly on yes-instances and none on no-instances.	
Polynomial-time reduction	Transforms source instances to target instances and transfers a fast target solver back to the source.	$A \rightarrow B$ means B is at least as hard
NP-hard	Every problem in NP reduces to the target; the target need not itself be in NP.	hardness requires reduction evidence
Computational-statistical gap	The data support the target accuracy, but an efficient computation is unavailable under the same assumptions.	information $\neq$ implementation

Optional deep dive

OPTIONAL DEEP DIVE - 0 CLASS MINUTES

Question: What must a detailed learning-problem reduction contain beyond the core definition?

Answer.

A detailed reduction must specify the instance map, prove that its encoded size and construction time are polynomial, and prove both directions of answer preservation. For a learning or optimization target it must also state exactly which decision version is being reduced to, or explain how a target optimizer decides that version. Shalev-Shwartz and Ben-David use this pattern with the **Maximum Feasible Subsystem** (Max FS) optimization problem. Its threshold decision version asks whether at least k inequalities can be satisfied simultaneously; an exact Max FS optimizer decides that question by comparing its optimum with k . Their reduction relates this optimization problem to empirical risk minimization for halfspaces: inequalities become labeled examples, feasible choices correspond to classifiers, and the number of satisfied constraints matches the number of correctly classified examples.

Omitting the size bound leaves open an exponentially expensive mapping; proving only one answer direction can create false positives or false negatives; and presenting only an exponential search algorithm says nothing about the existence of a different efficient algorithm. The core lesson teaches these proof obligations. Carrying out every coordinate of a particular reduction remains optional.

Post-class or self-study. Not a prerequisite for the core path.

Source notes

- M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*, 2nd ed., Appendix B.2–B.3, printed pp. 415–421 / PDF pp. 432–438: convex sets, convex functions, first-order characterization, and constrained optimization context.
- S. Shalev-Shwartz and S. Ben-David, *Understanding Machine Learning: From Theory to Algorithms*, Sections 12.1.1 and 12.1.3, printed/PDF pp. 156–163, and Sections 14.1–14.2, printed/PDF pp. 185–191: convexity, tangent and subgradient lower bounds, smoothness, the quadratic upper bound, and the unconstrained gradient update.
- E. Hazan, *Introduction to Online Convex Optimization*, 2nd ed., Sections 2.1–2.1.2, printed pp. 17–21 / PDF pp. 39–43: convexity, subgradients, Hessian curvature, smoothness, and convex-constrained first-order geometry.
- Shalev-Shwartz and Ben-David, Chapter 8, printed/PDF pp. 101–110: encoded inputs, efficient learning, exhaustive search, and the separation between sample and computational complexity.
- Cornell CS 4820, *Complexity Theory and NP-Hardness*, Spring 2025: standard definitions of P, NP, polynomial-time reductions, and NP-hardness, including the direction of the reduction implication.
- DSAA3073 Week 1 Learning Sheet v4, pp. 23–41, used for course scope, motivating questions, and sequencing. The textbook sources above control the mathematical statements; unsupported or overbroad claims in the historical sheet are not carried forward.